SOLUTION REQUIREMENTS

Date: 11 July 2026
Team ID: XXXXXX
Project Name: EduGenie – Google Gemini Powered Learning Assistant
Maximum Marks: 4 Marks

Define Functional and Non-Functional Requirements

This document defines the functional and non-functional requirements for **EduGenie – Google Gemini Powered Learning Assistant**. Functional requirements describe the features and services provided by the system, while non-functional requirements specify the quality, performance, security, reliability, and usability expectations.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	User Authentication	Secure login and user session management for students and teachers.	High
2	AI Question Answering	Answer academic questions using Google Gemini 1.5 Pro and LaMini-Flan-T5 models.	High
3	Concept Explanation	Generate clear and easy-to-understand explanations for academic concepts.	High
4	Quiz Generation	Automatically generate quizzes and multiple-choice questions based on the selected topic.	High
5	Text Summarization	Summarize lengthy educational content into concise notes.	High
6	Personalized Learning Path	Recommend study plans and learning resources based on student interests and learning progress.	High
7	API Integration	Integrate Google Gemini API with the FastAPI backend for AI-powered educational services.	High
8	Dashboard & History	Display previous learning activities, generated content, and recommendations.	Medium

---

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Generate AI responses quickly for all educational tasks.	Response within **5 seconds** (excluding AI model latency).
2	Scalability	Support multiple users accessing the application simultaneously.	Support **100+ concurrent users**.
3	Security & Data Privacy	Protect user information and learning history using secure communication.	HTTPS communication and encrypted data storage.
4	Reliability & Availability	Ensure continuous availability of the application.	**99% system uptime**.
5	Usability & Accessibility	Provide a responsive and user-friendly interface for students and teachers.	Compatible with desktop, tablet, and mobile devices.
6	Maintainability & Portability	Design the application with modular Python code and reusable components for easy maintenance and deployment.	Modular architecture with reusable FastAPI modules and easy deployment across platforms.